

HPC Tools for AI

BERT/SQuAD Single-GPU Baseline

Juan-Manuel TOUYA TALAVERA

Platform: CESGA FinisTerae III
Model: `google-bert/bert-base-uncased`
Dataset: SQuAD v1.1
Framework: PyTorch + Hugging Face Transformers
Hardware: 1 × NVIDIA A100-PCIE-40GB

Abstract

This report establishes a reproducible single-GPU reference baseline for fine-tuning BERT on SQuAD before moving to distributed training. The study focuses on training throughput, GPU utilization, memory usage, precision modes, batch size, DataLoader configuration, `torch.compile`, and timing stability on exactly one NVIDIA A100 GPU. The final performance baseline uses BF16, a per-device batch size of 160, and one full SQuAD epoch. Across three valid repetitions, the mean training time was **234.58 s** with a **0.42% coefficient of variation**. A separate quality-oriented run reached **79.56 Exact Match** and **87.50 F1** on SQuAD validation.

1 Objectives

The purpose of this baseline is to characterize BERT fine-tuning performance on a single accelerator before introducing distributed execution. The experiment was designed to fine-tune `google-bert/bert-base-uncased` on SQuAD v1.1, run on exactly one NVIDIA A100 GPU, measure the wall-clock duration of the training phase, identify relevant bottlenecks, and provide a reproducible reference for future distributed-training experiments.

An initial pilot series was discarded because the SLURM configuration exposed both GPUs of a two-A100 node to the training process. Those measurements are retained only for diagnostic purposes and are not included in the valid single-GPU results.

2 Experimental Environment

2.1 Software stack

The dedicated virtual environment is stored at `~/HPCTools/HPCTools4AI/.venv`.

2.2 Dataset preprocessing

The full SQuAD training split contains 87,599 original examples and produces 88,492 tokenized training features with the adopted sliding-window preprocessing. The maximum sequence length is 384 tokens, the document stride is 128, and fixed padding to the maximum sequence length is used.

Component	Version
Python	3.10.8
PyTorch	2.11.0+cu128
CUDA build	12.8
Transformers	5.17.0
Datasets	5.0.1
Accelerate	1.15.0
GPU	NVIDIA A100-PCIE-40GB

3 Final Baseline Configuration

Parameter	Value
Model	<code>google-bert/bert-base-uncased</code>
Dataset	SQuAD v1.1
GPU	$1 \times$ NVIDIA A100-PCIE-40GB
Precision	BF16
Epochs	1
Per-device batch size	160
Learning rate	3×10^{-5}
DataLoader workers	0
Gradient accumulation	1
Optimizer	Fused AdamW
Scheduler	Linear
<code>torch.compile</code>	Disabled
TF32	Default PyTorch behavior

One complete epoch already takes substantially more than one minute, so a one-epoch benchmark satisfies the workload-duration requirement while keeping repeated measurements practical.

4 Single-GPU Enforcement on CESSA

The official SLURM configuration requests one A100 and 32 CPU cores. The Python process is launched through an explicit `srun` step requesting one A100. Earlier pilot jobs using node-level `-exclusive` exposed both A100 GPUs of a two-GPU node to the batch shell, allowing the training framework to see two accelerators.

The final implementation therefore uses two safeguards: GPU isolation at the `srun` step and a runtime check in `train.py` that aborts unless exactly one CUDA device is visible.

5 Timing Methodology

The principal benchmark measures only the wall-clock duration of `trainer.train()`. CUDA synchronization is executed immediately before and after the timed region.

The reported time excludes model loading, dataset download/loading, tokenization, preprocessing, and final model serialization. This isolates the fine-tuning phase and improves comparability between repetitions.

6 Performance Optimization Study

6.1 Batch size

With the original precision configuration, increasing batch size from 48 to 96 and 128 did not materially improve throughput, while GPU memory usage increased substantially. After enabling BF16, batch size 160 was selected as the practical throughput/memory knee: batch 192 provided essentially no additional speedup while consuming more GPU memory.

6.2 DataLoader workers

Using 0, 2, or 4 workers produced nearly identical timings, around 59.2 s on the controlled subset. The input pipeline was therefore not a meaningful bottleneck, and `num_workers=0` was retained.

6.3 BF16

Precision	Mean time	Peak allocated memory
Default precision	~59.64 s	~12.46 GiB
BF16	~12.26 s	~8.61 GiB

BF16 delivered an approximate **4.87×** **speedup** and reduced peak allocated GPU memory.

6.4 TF32

Explicit TF32 reduced the default-precision mean training time from approximately 59.63 s to 19.55 s. BF16 nevertheless remained faster and more memory-efficient, so BF16 was selected for the official baseline.

6.5 `torch.compile`

Mode	Mean training time
Eager	~87.30 s
Compiled	~151.31 s

For the tested short workload, compilation overhead dominated and made the compiled path slower. A longer three-epoch experiment reduced the relative penalty but did not reverse the overall result. A recompile was also observed for the irregular final batch. Therefore, `torch.compile` was disabled for the reference run.

7 Profiling Results

The sustained GPU utilization and the absence of measurable gains from additional DataLoader workers support the conclusion that the timed training phase is primarily GPU-bound.

Metric	Representative value
Mean GPU utilization	~89.6%
GPU utilization $\geq 80\%$	~97.6% of GPU samples
Peak GPU memory	~26.8 GiB
Mean GPU power	~238 W

Repetition	Training time
1	233.48 s
2	235.41 s
3	234.85 s

8 Official Baseline Results

The final single-A100 baseline is approximately **234.6 seconds, or 3.91 minutes, per full SQuAD epoch**. The low coefficient of variation indicates good run-to-run stability.

9 Quality-Control Experiment

The throughput-oriented baseline is not intended to maximize machine-learning quality. A separate quality-oriented run used the full SQuAD training data, batch size 12, two epochs, learning rate 3×10^{-5} , approximately 10% warmup, BF16, fused AdamW, weight decay 0.01, and validation after each epoch.

Validation loss was 0.9823 after epoch 1 and 0.9893 after epoch 2, so the epoch-1 checkpoint was selected as the best model.

This experiment highlights the distinction between HPC throughput optimization and machine-learning generalization. Because several hyperparameters changed simultaneously, the quality improvement should not be attributed to any single factor in isolation.

10 Reproducibility

All valid performance runs enforce exactly one visible CUDA GPU. The random seed used for the experiments is 42. Small numerical differences may remain because GPU kernels and framework internals are not guaranteed to be bitwise deterministic.

Generated results, checkpoints, TensorBoard files, virtual environments, and model weights are intentionally excluded from Git. Detailed experimental measurements are documented in `EXPERIMENTS.md`.

11 Conclusion

The final reference configuration is BERT-base-uncased on full SQuAD v1.1, using one NVIDIA A100-PCIE-40GB, BF16, batch size 160, and one epoch. The mean training time is approximately **234.6 s** with a **0.42% coefficient of variation**.

BF16 was the most important optimization, delivering an approximate **4.87 $\times$ speedup** on the controlled subset. DataLoader parallelism did not provide measurable benefit, while `torch.compile` was not advantageous for the short benchmark because compilation overhead dominated.

Statistic	Value
Mean	234.58 s
Median	234.85 s
Sample standard deviation	0.99 s
Coefficient of variation	0.42%

Metric	Score
Exact Match	79.56
F1	87.50

The separate quality-oriented run achieved **79.56 EM / 87.50 F1**. Together, these results provide a reproducible single-GPU HPC reference and a machine-learning quality sanity check for the distributed-training phase.